RAPID MVP PROJECT BLUEPRINT

AI-Enabled Predictive Maintenance for Government Vehicles & Assets

A focused implementation guide for building a credible, demo-ready predictive-maintenance prototype from existing asset, maintenance and inspection records - with the scope deliberately limited to what can be completed fast and explained clearly.

09

MINECOFIN | Predictive Maintenance | Rapid MVP | One Public Institution

Prepared from the Tech Titans Problem Framing Canvas
Workshop source dated 26 Sept 2026, UniPOD

Rapid MVP edition
29 Sept 2026

DOCUMENT CONTROL

How to use this rapid MVP blueprint

This edition keeps the validated MINECOFIN problem and research foundation, but converts the implementation plan into a tightly scoped prototype that can be built, tested and demonstrated quickly.

BASELINE

Source-derived facts

- Challenge owner: MINECOFIN.
- Problem: fixed maintenance schedules do not reflect actual asset condition.
- Primary users: asset/fleet managers and maintenance staff.
- Key data: asset register, maintenance logs, inspection and inventory records.
- Pilot scope: one public institution.

EXECUTION

Rapid build principle

- Use approved exports or demo files - not live-system integration.
- Prefer one or two asset classes.
- Build batch risk scoring, not real-time infrastructure.
- Use a simple model and a clear dashboard.
- Optimize for a working end-to-end demonstration.

LATER PHASE

Deferred beyond MVP

- Production-grade system integration.
- Hardware or telemetry retrofits.
- Automated repair authorization.
- Enterprise identity and deployment hardening.
- Nationwide rollout and procurement packaging.

Decision rule: any feature that does not directly help demonstrate *data -> risk score -> explanation -> maintenance action* is deferred from this edition.

EXECUTIVE SUMMARY

The project in one page

The fastest credible solution is a small decision-support prototype: ingest an approved dataset, calculate maintenance risk, explain why an asset is flagged, and present prioritized actions in a simple dashboard.

1
pilot institution
controlled scope

3
build days
rapid MVP sprint

1-2
asset classes
keep data homogeneous

80%
early-detection target
stretch/offline if data permits

1
working demo
end-to-end decision loop

What the prototype does

Loads asset and maintenance records, cleans them, creates transparent features, assigns a risk score and ranks assets for preventive inspection.

What the user sees

A dashboard with fleet risk summary, priority queue, asset details, reasons for risk and a recommended inspection action.

What success means now

The prototype works end-to-end, produces defensible risk rankings, shows useful explanations, and can be demonstrated reliably using real approved or clearly labeled demo data.

Important: the workshop's 30% downtime and 20% maintenance-cost reduction targets remain institutional pilot goals. They cannot be causally proven during a short prototype build; this edition focuses on technical feasibility and decision usefulness.

CONTENTS

Chapters & navigation

The book is organized around a rapid prototype: validate the problem, narrow the data, build the model, create the decision interface, test the demo and define the next step.

01 Case & validated problem p. 5

02 Success hierarchy p. 7

03 Users & current process p. 8

04 Rapid MVP scope p. 11

05 Data & labels p. 13

06 Target workflow & architecture p. 16

07 Model & features p. 20

08 Dashboard & experience p. 24

09 Three-day execution plan p. 25

10 Team, QA & demo p. 28

11 Risk & transition p. 31

12 References & appendices p. 33

CHAPTER 01 - CASE

Challenge #9: MINECOFIN

The case is a practical AI decision-support challenge: identify government vehicles and assets that need preventive maintenance before breakdowns occur.

#9 MINECOFIN (Recommended)

PROBLEM

Government vehicles & assets are maintained on fixed schedules rather than actual condition.

AI SOLUTION

AI-powered predictive maintenance model to flag assets needing preventive maintenance.

KEY DATA

IFMIS asset register, maintenance logs, inspection & inventory records.

PILOT SCOPE

One public institution.

CASE

Problem

Government vehicles and assets are maintained on fixed schedules rather than actual condition.

CASE

AI solution

Predictive maintenance model to flag assets needing preventive maintenance.

CASE

Key data

IFMIS asset register, maintenance logs, inspection records and inventory records.

CASE

Pilot scope

One public institution.

Rapid interpretation: the strongest demonstration is not an elaborate platform. It is a trusted workflow that turns existing records into a prioritized list of assets, explains the risk, and gives staff a clear next action.

CHAPTER 01 - PROBLEM FRAMING

Validated problem statement

The workshop already establishes the core users, process gap, root causes, constraints and success boundaries. The rapid MVP should not reopen the problem definition unless data contradicts it.

> “For government asset managers and maintenance officers, the reliance on fixed maintenance schedules and manual inspections makes it difficult to identify vehicles and equipment that need maintenance, resulting in unexpected breakdowns, increased repair costs, and prolonged asset downtime.”

Primary users

- Fleet/asset managers
- Maintenance officers/technicians
- Drivers/operators as issue reporters

Root causes

- Schedule-driven decisions
- Maintenance data underused for prediction
- Limited forward-looking risk prioritization

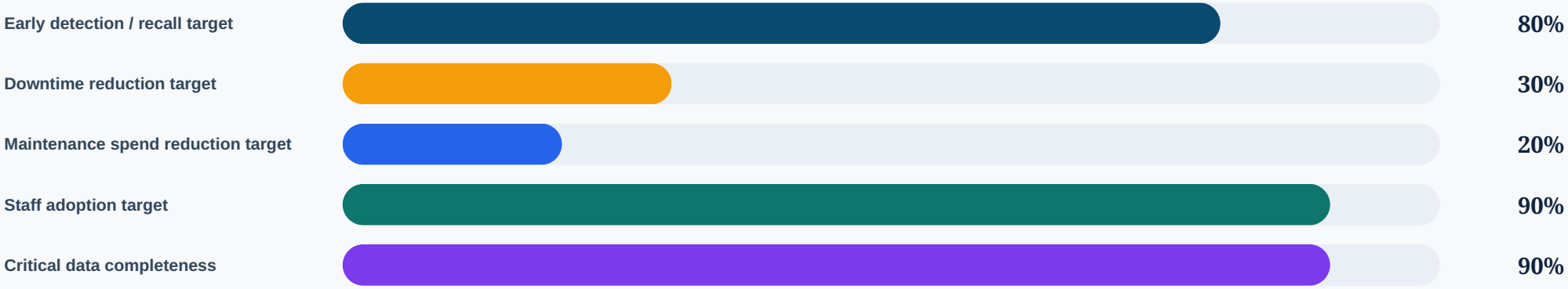
Prototype answer

- Historical data summarized into features
- Risk-ranked asset queue
- Reason codes + recommended inspection action

CHAPTER 02 - SUCCESS

Separate long-term institutional outcomes from rapid MVP proof

This distinction protects the team from overclaiming. The prototype proves that the decision loop works; a later operational pilot measures downtime and cost impact.



Targets originate from the workshop. Operational impact targets require a real pilot observation period.

Must prove now

End-to-end data ingestion, scoring, explanation and dashboard flow works without manual patching during the demo.

Should prove now

Risk ranking is better than a simple random/list baseline and the reasons shown to users are understandable.

Prove later

Actual reduction in breakdowns, downtime and maintenance expenditure using a longer controlled institutional pilot.

CHAPTER 03 - USERS

Stakeholder map for the prototype

Keep roles few and explicit. One person must own the problem, one must explain the data, one must validate maintenance logic, and the technical team must build the prototype.

Role	Who	Rapid-MVP accountability
Challenge / product owner	Fleet or asset-management representative	Confirms use case, chooses priorities, validates alert usefulness
Domain SME	Maintenance officer / technician	Defines what counts as a meaningful fault and whether explanations are actionable
Data contact	Asset/maintenance data custodian	Provides approved extract and explains fields/codes
Data & AI	Data/ML developer(s)	Clean data, engineer features, train/evaluate model
App / UX	Frontend or full-stack developer	Build simple dashboard and demo flow
Presenter / PM	Team lead	Controls scope, integration, QA, story and final presentation

Operating rule: do not wait for perfect organizational mapping. For the prototype, every unresolved question must have one named owner and a same-day decision deadline.

CHAPTER 03 - PROCESS

Current-state maintenance flow

The existing logic is mainly schedule/history based. The prototype adds a risk-prioritization step before inspection; it does not replace the existing maintenance process.

Driver / operator					
Use asset Daily operation	**Observe** Noise, warning, fault	**Report** Issue if noticed	**Wait** Schedule/inspection	**Return** After repair	**Repeat** Cycle

Fleet / asset manager					
Register Asset record	**Schedule** Time/usage interval	**Review** Issues/history	**Prioritize** Urgency	**Approve** Maintenance	**Track** Status/cost

Maintenance team					
Inspect Scheduled/reactive	**Diagnose** Defect	**Repair** Parts/labor	**Record** Work done	**Close** Work order	**History** Stored

Decision gap: historical records exist, but they are not systematically converted into a forward-looking priority score before failure.

CHAPTER 03 - DIAGNOSIS

Root causes that matter for a fast prototype

The prototype should address only the causes that can materially affect the demo: inconsistent IDs, weak labels, missing fields, and an unclear action after a risk alert.

Identity

Asset identifiers may differ across files or logs. Fix this first; nothing else is reliable without linkage.

Labels

Scheduled service and corrective failure may be mixed. Create a simple event codebook.

Data gaps

Usage, inspection severity or costs may be incomplete. Use explicit missing-data flags.

Action gap

A score without a recommended inspection action is not operationally useful.

Scope discipline: do not solve every government data-governance or enterprise-integration issue inside the prototype. Capture them as next-phase requirements while building a clean, explainable demo dataset now.

CHAPTER 04 - RAPID MVP

Prototype charter: one institution, one decision loop

The prototype is complete when one approved dataset can move through a repeatable pipeline and produce a risk-ranked queue with explanations and a recommended inspection action.

In scope

- One public institution or one representative demo dataset.
- One or two comparable asset classes.
- CSV/Excel/approved data extract ingestion.
- Cleaning, feature generation and risk scoring.
- Rules baseline + one simple ML model.
- Simple dashboard: overview, queue, asset detail.
- Reason codes and human action recommendation.
- Offline metrics and demo audit trail.

Not part of the rapid MVP

- Live IFMIS production integration.
- Real-time streaming or hardware retrofits.
- Automated maintenance approval.
- Complex spare-parts optimization.
- Chatbot or generative-AI assistant.
- Enterprise SSO, high-availability deployment or disaster recovery.
- Formal nationwide rollout or procurement execution.
- Deep-learning research or multi-model ensembles.

CHAPTER 04 - SCOPE CONTROL

What to build - and what to cut immediately

A three-day prototype fails mainly through scope creep. Use the filter below before accepting any new feature.

Scope question	Decision rule
Directly demonstrates core value?	Keep if it improves data -> risk -> explanation -> action.
Can it be completed and tested in hours?	Keep. If it needs new infrastructure, defer.
Needs access/approval the team does not already have?	Replace with approved file input or demo mode.
Can judges/users see its benefit in under 30 seconds?	Prioritize.
Requires a new backend service or unfamiliar framework?	Avoid unless already working.
Is it only “nice to have”?	Cut.

Core deliverable

A stable dashboard that ranks assets by maintenance risk and explains the ranking.

Proof deliverable

A metrics page/notebook showing how the model was evaluated against a simple baseline.

Presentation deliverable

A short, rehearsed live demo plus a screen-recorded backup.

CHAPTER 05 - DATA

Minimum dataset for the prototype

Use the smallest field set that can create a defensible maintenance-risk signal. Extra columns are optional unless they improve the label or explanation.

Data domain	Minimum fields	Why it matters	Priority
Asset master	asset_id, class/type, make/model, year/acquisition date	Identity + age/context	Required
Maintenance	event_date, planned/corrective flag, fault/description, cost, close date	Outcome/history	Required
Inspection	inspection_date, defect/severity, outcome	Early condition signal	High
Usage	odometer or engine hours	Exposure since service	High if available
Downtime	out-of-service start/end or downtime hours	Operational impact	Useful
Inventory	major part replaced / part code	Repair context	Optional

Fast rule: if a field cannot be cleaned or explained quickly, leave it out of the first model. A smaller trustworthy table beats a large inconsistent one.

Rapid data check before modeling

Run these checks immediately. Any blocker that cannot be solved quickly triggers demo-mode fallback rather than a prolonged integration effort.

Dimension	Fast question	Rapid gate	Status
Identity	Can maintenance/inspection rows link to one asset ID?	>=95% for prototype	Blocker
Label	Can corrective/unplanned events be separated from routine service?	Yes / SME-approved simple rule	Blocker
Completeness	Are the main feature fields sufficiently populated?	Measure and expose missingness	Manage
History	Are there enough past events to learn patterns?	Prefer multiple months/cycles	Model-dependent
Class balance	Are there enough corrective events?	Use class weights / simpler rules if rare	Manage
Plausibility	Dates, costs and odometer/hours make sense?	No impossible critical values	Blocker

Red flag: if almost all maintenance rows are scheduled services, a classifier can accidentally learn the calendar rather than failure risk. Keep scheduled work separate from corrective/unplanned events.

CHAPTER 05 - LABELS

Define the target - and a safe fallback if real labels are weak

The prototype needs a clear outcome. If historical corrective-event labels are inadequate, demonstrate a transparent risk-ranking model rather than pretending to predict failure accurately.

Preferred supervised target

Will this asset experience a qualifying corrective/unplanned maintenance event within the next 30 or 60 days? Qualifying events are defined with the maintenance SME.

Fallback risk score

If labels are weak, combine transparent signals such as repeated repairs, severe inspection findings, high usage since service, recent corrective cost and asset age into a weighted risk score.

MODE A

Real approved data

Best option. Keep source lineage and document cleaning decisions.

MODE B

Partial data

Use real fields that are reliable; clearly mark unsupported features as unavailable.

MODE C

Demo dataset

Use simulated/synthetic records when no approved data is available. Label the dataset as simulated and do not present the metrics as institutional results.

CHAPTER 06 - TARGET WORKFLOW

The rapid “to-be” decision loop

Batch scoring is enough. The prototype should make one complete loop easy to understand and repeat.

1. Load
CSV/Excel or approved extract

›

2. Clean
IDs, dates, codes, missingness

›

3. Features
Recency, frequency, severity, usage, cost

›

4. Score
Rules + simple model

›

5. Explain
Top reason codes

›

6. Prioritize
Critical / High / Watch / Normal

›

7. Act
Inspect / monitor / verify data

›

8. Record
Disposition for demo/audit

Batch first

A manual “Refresh scores” button is acceptable for the prototype. Avoid scheduled infrastructure.

Human decision

The system recommends inspection priority; authorized staff remain responsible for maintenance decisions.

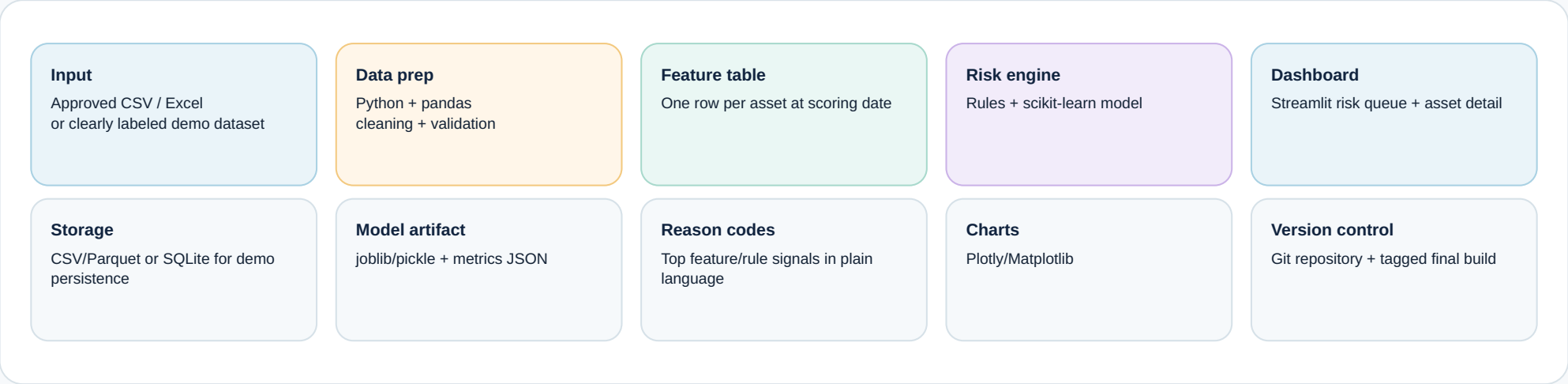
Visible evidence

Every risk flag should show the specific data signals that raised the score.

CHAPTER 06 - ARCHITECTURE

Simplified reference architecture for the rapid MVP

Use one local or hosted application with direct file ingestion. Avoid microservices and separate APIs unless the team already has them working.



Design objective: one command launches the app, one file refreshes the dataset, and one button or automatic page load updates the risk ranking.

CHAPTER 06 - TECHNOLOGIES

Technology stack optimized for speed

Choose familiar, low-friction tools. Do not introduce a platform merely because it is “enterprise-grade.”

Layer	Recommended	Why it fits rapid MVP
Language	Python 3.x	One language for data, model and app logic
Data	pandas + NumPy	Fast tabular cleaning and feature engineering
Model	scikit-learn	Logistic Regression + Random Forest / HistGradientBoosting
Persistence	CSV/Parquet; SQLite optional	No database setup required for basic demo
Dashboard	Streamlit	Very fast dashboard, filters and forms
Charts	Plotly or Matplotlib	Risk distributions, feature summaries and KPI visuals
Model save	joblib	Simple model serialization
Version control	Git / GitHub	Team collaboration and final tagged build

Optional only if already working: XGBoost/LightGBM, cloud hosting, database server, external APIs or advanced explanation libraries. None is required to prove the concept.

CHAPTER 06 - DATA PREPARATION

Data pipeline that can be completed quickly

The goal is a reproducible clean table, not an enterprise data platform. Save every transformation in code so the demo can be rerun.

1. Ingest
Read files; snapshot originals

›

2. Standardize
Column names, dates, IDs, units

›

3. Validate
Duplicates, nulls, impossible values

›

4. Join
Asset + maintenance + inspection

›

5. Derive
Features and target

›

6. Export
model_ready.parquet/csv

Fast validation

Print row counts before/after joins, unique asset counts, positive-label rate and missingness.

Leakage guard

Features for a scoring date may use only information known before that date.

Demo reproducibility

One script/notebook should regenerate the model-ready dataset from the raw demo files.

CHAPTER 07 - PREDICTIVE ANALYTICS

Model strategy: simple, comparable, explainable

Use a transparent baseline and one stronger model. Stop when the stronger model adds enough value to justify its complexity.

BASELINE

Level 0 - Rules

Weighted rules using repair frequency, inspection severity, recent cost, usage since service and asset age. Always build this first.

MODEL A

Level 1 - Logistic Regression

Good audit baseline, fast to train, easy to calibrate and explain. Handles class weights.

MODEL B

Level 1 - Tree model

Random Forest or HistGradientBoosting captures non-linear interactions while staying fast on tabular data.

Selection rule: choose the model that best balances recall, precision, stability and explanation quality. Do not choose the most complex algorithm by default.

CHAPTER 07 - FEATURES

Minimum feature blueprint

Use a short list of features maintenance staff can understand. Most should be simple rolling counts or recency measures.

Feature family	Examples
Recency	Days since last service; days since last corrective repair; days since inspection
Frequency	Repairs in 30/90/180 days; repeated fault count
Severity	Latest/max inspection severity; safety-critical defect flag
Usage	Distance or engine-hours since service; recent usage rate
Cost	Corrective cost in recent period; high-cost repair flag
Age/context	Asset age; class/type; make/model if sample size allows
Data quality	Missing usage flag; stale inspection flag; inconsistent odometer flag

Keep the model class-aware: a truck, motorcycle and generator should not share identical thresholds or “normal” utilization patterns without normalization.

CHAPTER 07 - EVALUATION

Evaluate usefulness, not accuracy alone

Maintenance failures may be rare. The model must catch meaningful events without overwhelming users with false alarms.

>=80%	Track	Track	Count
Recall	**Precision**	**PR-AUC**	**Alerts/day**
stretch target if the dataset supports a valid label	how many alerts are actually useful	ranking quality for rare positives	must fit realistic review capacity

Validation design

- Prefer time-based split when dates are reliable.
- Compare rules, logistic regression and tree model on the same holdout.
- Report confusion matrix and alert volume.
- If data is simulated, label all metrics as demo metrics.

Model acceptance

- No obvious leakage.
- Explanations are plausible.
- Risk ranking separates higher- and lower-risk assets.
- Prototype remains stable when the file is refreshed.

CHAPTER 07 - EXPLAINABILITY

Turn scores into plain-language reasons

A judge or maintenance officer should understand a flagged asset in seconds. Reason codes are sufficient; advanced explanation tooling is optional.

Risk score

Show 0-100 or Low/Watch/High/Critical. Keep the threshold logic fixed for the demo.

Top reasons

Examples: “3 corrective repairs in 90 days”, “high defect severity”, “usage since service is high”, “repair cost rising”.

Recommended action

Examples: inspect within 48 hours, verify reported fault, check component group, confirm usage reading.

Guardrail: when data is missing or invalid, show “manual review - insufficient data” instead of producing a confident risk result.

CHAPTER 08 - PRODUCT EXPERIENCE

Three-screen dashboard is enough

Every screen should answer a maintenance decision. Avoid admin panels, deep settings and extra modules unless the core flow is already finished.

Screen 01 - Overview

- Asset count by risk band
- Top 5 high-risk assets
- Data-quality warnings
- Model/demo status badge

Screen 02 - Priority queue

- Asset ID + class
- Risk score/band
- Top reason
- Last service / last repair
- Recommended inspection action

Screen 03 - Asset detail

- Maintenance timeline
- Key feature values
- Risk reasons
- Simple cost/repair trend
- Action / disposition form

Design target: from the landing screen, a user should reach the highest-risk asset and understand why it is flagged in under 20 seconds.

CHAPTER 09 - EXECUTION

Day 1 - Data, target and first model

End the day with a clean model-ready dataset, a target definition, a rules baseline and at least one trained model.

Morning

Lock the data and problem

08:00-09:00	Team roles, repository, folder structure, final scope.
09:00-10:30	Inspect columns, IDs, event types, missing values and label candidates.
10:30-12:00	Create event codebook and decide supervised target or fallback risk score.

Afternoon

Build the data pipeline

13:00-14:30	Clean IDs, dates, duplicates, units and critical missingness.
14:30-16:00	Join tables and engineer minimum feature set.
16:00-18:00	Train rules baseline + Logistic Regression; save metrics and model artifact.

Exit gate

Do not carry blockers overnight

- One reproducible clean dataset.
- Asset ID linkage verified.
- Target/fallback method documented.
- Baseline metrics available.
- At least one model produces risk scores.
- Dashboard schema agreed.

If blocked: switch to clearly labeled demo data before the day ends.

Day 2 - Finalize model and build the dashboard

End the day with one working application that loads the data, scores assets and displays the priority queue and asset detail.

Morning

Select the model

08:00-09:30	Train one tree model and compare with rules/logistic baseline.
09:30-10:30	Choose thresholds; compute recall, precision and alert volume.
10:30-12:00	Create reason-code function and example asset explanations.

Afternoon

Build the user experience

13:00-14:30	Streamlit overview + KPI cards + risk distribution.
14:30-16:00	Priority queue with filters and risk reasons.
16:00-18:00	Asset detail view, maintenance timeline and action/disposition field.

Exit gate

Working end-to-end prototype

- App launches reliably.
- Scores load from the final pipeline/model.
- Top-risk assets are visible.
- Reasons are readable and plausible.
- At least one asset detail page works.
- No placeholder screens remain.

Freeze scope: no new features after this gate.

CHAPTER 09 - EXECUTION

Day 3 - Test, polish, rehearse and present

The final day is for reliability and storytelling. A smaller stable demo is stronger than a larger unstable build.

Morning

Break the app before judges do

08:00-09:30	Run full pipeline from raw input to dashboard; fix errors.
09:30-10:30	Test missing values, duplicate rows, unsupported asset and empty results.
10:30-12:00	Polish labels, charts, typography and demo sample ordering.

Afternoon

Pitch + backup

13:00-14:00	Freeze final code and dataset; tag repository.
14:00-15:00	Record 2-3 minute backup video of the working demo.
15:00-17:00	Rehearse presentation and live demo repeatedly.
17:00+	Final machine, power, browser, dataset and offline backup check.

Exit gate

Demo-ready package

- Stable final app.
- Frozen demo dataset.
- Metrics screenshot / notebook.
- Presentation deck aligned with demo.
- Backup screen recording.
- Repository + README.
- One presenter and one technical backup ready.

CHAPTER 10 - TEAM

Parallel work without integration chaos

Split the work so data/model and application can progress in parallel, but force integration checkpoints twice per day.

Lead / PM

Own scope, issue log, decisions, final integration and presentation story.

Data / ML

Own cleaning, label, features, baselines, model, metrics and saved scoring output.

App / UX

Own Streamlit pages, charts, queue, asset detail and user actions.

Domain / QA

Validate maintenance logic, test outputs, collect screenshots and challenge weak assumptions.

09:00 sync

What will be finished by lunch? Identify blockers immediately.

13:00 integration

Model output schema and app schema must match. Merge working code.

18:00 gate

Run the complete user flow on one machine. No “it works on my laptop” acceptance.

CHAPTER 10 - QUALITY

Minimum QA checklist for a credible demo

Test the failure modes most likely to embarrass the team: broken joins, stale model files, empty filters, impossible scores and application crashes.

Test area	Minimum evidence
Data	Duplicate asset IDs, missing key fields, invalid dates, impossible usage values
Model	Correct feature order, no leakage, output between expected bounds, baseline comparison saved
App	Loads cleanly, filters work, queue sorts correctly, asset detail matches selected row
Explanations	Reason codes match underlying feature values; no contradictory statements
Fallbacks	Missing-data asset shows manual-review message; empty dataset handled gracefully
Demo	Works offline/local if network fails; fixed demo input available; browser cache cleared
Reproducibility	README contains exact launch command and environment/package list

Release rule: after the final freeze, only fix defects that can break the core demo. Do not add features.

CHAPTER 10 - PRESENTATION

Live demo and pitch runbook

The story should move from public-sector pain to a visible decision in under five minutes. Keep the live interaction deterministic.

1. **Problem**
   Government assets are maintained by schedule; historical maintenance data is underused for early risk detection.
2. **Data**
   Show the minimum fields and that the system uses existing records rather than new hardware.
3. **Overview**
   Open the dashboard and show risk distribution and the highest-risk assets.
4. **Asset**
   Click one high-risk asset; explain the score using 2-3 plain-language reasons.
5. **Action**
   Show recommended inspection action and how a user records the outcome/disposition.
6. **Evidence**
   Show offline recall/precision or risk-ranking validation and clearly state whether data is real or simulated.
7. **Close**
   Explain the path from prototype to an institutional pilot and how impact would be measured.

Backup plan: keep a 2-3 minute screen recording and key screenshots ready. If the live app fails, continue the story without stopping.

CHAPTER 11 - RISK

Rapid risk register and contingency plan

The highest risks are data availability, scope creep and demo instability. Each needs a predefined fallback.

ID	Risk	Impact	Immediate mitigation
R1	No approved real dataset	High	Switch to clearly labeled simulated demo dataset; preserve the same schema.
R2	Corrective labels unusable	High	Use transparent weighted risk score and present supervised model as future validation.
R3	Too few positive events	Medium	Use ranking/score demonstration; avoid misleading classification claims.
R4	Model weak or unstable	Medium	Use rules or logistic baseline; focus on workflow value and explanation.
R5	Dashboard takes too long	High	Cut to overview + queue + asset detail only.
R6	Team merge conflicts	Medium	One integration branch; fixed score-output schema; merge twice daily.
R7	Network/cloud failure	High	Run locally; keep package environment and data offline.
R8	Live demo failure	High	Use frozen build + backup screen recording/screenshots.

Best risk response: simplify. The prototype is judged by whether the core decision loop is believable and working, not by the number of technologies used.

CHAPTER 11 - TRANSITION

What happens after a successful prototype

The rapid build should end with a clear handoff path. Production work is a separate phase and should be justified by evidence from the prototype.

Prototype
Working demo + documented data schema

›

Validation sprint
Authorized real data + SME label review

›

Institutional pilot
One institution, controlled workflow, measured adoption

›

Impact evaluation
Downtime, cost, prevented failures, user trust

›

Scale decision
Integrate, harden and expand only if benefits justify it

Next evidence

Exact source fields, data volume, label quality, users and operational baseline.

Next engineering

Approved integration path, authentication, hosting, monitoring, security and support.

Next business case

Validated reduction in reactive maintenance, downtime and emergency repair spend.

CHAPTER 12 - REFERENCES

Primary sources retained for the blueprint

The rapid edition uses the workshop as the project baseline and retains selected Rwanda public-sector sources from the prior research edition for institutional context.

ID	Reference	Source / URL
R1	Tech Titans Problem Framing Canvas	Workshop source - 26 Sept 2026, UniPOD
R2	MINECOFIN - IFMIS	https://www.minecofin.gov.rw/mandate/projects/ifmis
R3	Smart IFMIS - Asset & Inventory	https://www.smartewp.minecofin.gov.rw/
R4	MINICT - National Artificial Intelligence Policy	https://www.minict.gov.rw/ai-policy
R5	RISA - Digital Adoption Implementation Guidelines: Data	https://guidelines.risa.gov.rw/books/digital-adoption-implementation-guidelines/page/data
R6	NCSA - Cybersecurity documentation	https://cyber.gov.rw/documentation/
R7	DPO - Law No. 058/2021 on Personal Data and Privacy	https://dpo.gov.rw/dpp-law/general-provisions
R8	NIST - AI Risk Management Framework	https://www.nist.gov/itl/ai-risk-management-framework

Research discipline: this rapid edition does not claim that a production IFMIS predictive-maintenance interface already exists. Production integration must be validated separately.

APPENDIX A

Rapid MVP data dictionary

A minimal schema the team can use immediately for file-based ingestion and dashboard development.

Field	Type	Meaning	Status
asset_id	String	Stable unique identifier	Required
asset_class	Category	Comparable class/type	Required
make_model	Category	Make/model or equipment type	Preferred
asset_age_years	Numeric	Age at scoring date	Derived
event_date	Date	Maintenance/inspection event date	Required
event_type	Category	scheduled / corrective / inspection / breakdown	Required
fault_or_defect	Text/category	Fault or defect description/code	Preferred
severity	Ordinal	Inspection defect severity	Preferred
usage_value	Numeric	Odometer or engine-hours	If available
maintenance_cost	Numeric	Corrective/maintenance cost	If available
downtime_hours	Numeric	Out-of-service duration	If available
risk_score	Numeric	Prototype output 0-100	Output
risk_band	Category	Normal / Watch / High / Critical	Output
reason_1..3	Text	Plain-language top reasons	Output

APPENDIX B

Scoring logic and final readiness checklist

Keep the scoring pipeline small enough to understand during the presentation and robust enough to rerun from raw input.

```
data = load_input_file()
clean = standardize_and_validate(data)
features = build_asset_features(clean)

if supervised_label_is_valid:
    risk = model.predict_proba(features)[:, 1]
else:
    risk = weighted_rule_score(features)

reasons = build_reason_codes(features, risk)
bands = assign_risk_band(risk)
results = publish_dashboard_table(risk, bands, reasons)
```

FINAL READINESS

- ✓ Raw input file frozen.
- ✓ Data cleaning script reruns without manual edits.
- ✓ Model/rule artifact saved and versioned.
- ✓ Risk queue contains expected assets.
- ✓ High-risk example has understandable reasons.
- ✓ Missing-data fallback tested.
- ✓ App launches from documented command.
- ✓ Demo works without network dependency.
- ✓ Backup video and screenshots ready.
- ✓ Presenter can complete story without technical detours.

Definition of ready: the team can move from a frozen input file to a visible, explained maintenance-risk decision in a repeatable demo without relying on unfinished infrastructure.